Random walks in multi-layered networks with applications on Zurich's bus and tram network

Florian Grifone

Department of Mathematics, UZH

This manuscript was compiled on March 14, 2025

Random walks are key stochastic processes that are used to study the dynamics of the network. Not only are they very natural to model real-life problems, as for example the spread of a disease in a city, but can also be further used to detect communities in a network or to measure the PageRank centrality of nodes. For single-layer undirected networks the simplest random walk can be constructed using the degree of the nodes (see (1) chapter 6). However no matter whether nodes represent people, cities, social groups or public transport stations, very often they belong to different networks at the same time. Treating these networks independently can lead to false conclusions, especially when trying to model dynamic processes. Thus if we want to model dynamics in which our nodes can belong to different layers of networks, then the random walk must also be encoded with a transition kernel that allows a change of layers. Depending on how realistically the random walk is supposed to imitate the dynamics in this multi-layered network, the construction of the transition kernel can be quite challenging. The goal of the research presented here, was to construct a random walk on the two layer network of bus- and tramlines of the city of Zurich (2), that would not only jump randomly from station to station, but rather mimic how a person might move along bus and tram lines. Then the quality of the public transport network would be measured by calculating the coverage, which is the average percentage of stations visited up to a specific time ((3)).

complex networks | multi-layered networks | random walks

Significance Statement

Random walks are a simple model to understand and programm, that can be used to simulate movement in a network of nodes. However movement of people is not random, as for example people do not generally drive forward and backward between two stations. Also a people using public transport generally do not restrict themselves to only one mean of transport, so why should the random walk? With this research we tried to code a simulation that keeps track of were it was up to a few steps ago and wether it changed between tram and bus already at a specific station, so that it does not get stuck between a few stations. This should mimic a person trying to reach as many stations as possible given a certain amount of time, while not influencing too much the decision of which route he is allowed to take. Then we try to evaluate the quality of the network by computing on average what fraction of the total amount of stations it has reached.

Introduction

To study dynamics of diffusion processes in a network, for example the spread of a disease in a community or the transportation of citizens through a public transport network, random walks are most commonly used to model them. On a graph, the random walk can be constructed by choosing a probability distribution at each node to choose a neighbor as a next step of the walk. Important is that this probability does not depend on other nodes, at each step. One possibility is to give each neighbor the same probability of being the next location. For a single-layered network random walks have been studied extensively, and there are many rigorous results about the long term evolution of the dynamics. However single-layer networks (especially unweighted and undirected ones) are often too simplistic to capture real life examples. As an example public transport networks for many cities consist of two or three different means of transportation that run on different networks. Also a random walk that just depends on the degree of the nodes, might not always result in the most realistic model.

Consider a network that has a tree-like shape, meaning that most nodes only have a degree of 2 and only few have a high degree. If in such a network the random walk at each node has a uniform distribution to jump to a neighbor, then there is non-negligible chance that the random walk will jump back and forward between two nodes that have a degree of 2, since the probability of this happening is 25%. Now if we wanted to simulate the movements of passengers in a public transport network, then jumping back and forward repeatedly between two bus stations does not represent a realistic behaviour of a passenger. Additionally many cities have two or three different means of public transport, and sometimes to move faster through the city it is more favorable to change between the options. So beside having the option of jumping to neighbors, the random walk should also include an event of changing the layer.

In (3) Manilo De Domenice et al defined different continuous time random walks and applied them on the three-layer public transport network for the city of London. To construct the transition kernel, they took a uniform cost at each node for the

Author affiliations: [1] Department of Mathematics, UZH

All work was done by F. Grifone. Neither code or report were written with the help of artificial intelligence.

option to change the layer (often the cost was taken as 1) and a weighted probability to jump to a neighbor of the current network layer. The normalising constant for each row of the transition kernel was the sum of the weighted probabilites and the cost. In the end they measured on average how much of the network the random walk had covered up to a fixed finishing time.

To model the diffusion process of passengers more realistically in our experiment we decided to adapt the cost depending on the degree of a node in the two different layers. For example if the degree k_i^2 at a node i in layer 2 is higher than the degree k_i^1 in layer 1, and the node is in layer 1, then the probability of changing to the layer 2 at i is proportional to the ratio $\frac{k_i^2}{k_i^1}$. Intuitively this is supposed to mimic the situation at a station connected by one bus line and four tram lines, in which case it would be more likely that we would change from bus to tram than from tram to bus. Additionally we decided to alter the transition kernel in a way that if we indeed changed from bus to tram then we are not allowed to change back to tram at this station anymore. This prevents that the random walk gets stuck changing layers repeatedly at the same station with high degree in both layers over and over again. We should keep in mind however, that in this case the random walk is not a Markov chain anymore (see (4)), more to this later. Moreover during the run of the random walk if he wants to immediately go back to the node he just came from, we force him to repeat the step until he goes to another node.

We applied our random walk on the network of bus and tram lines of the city of Zurich, and indeed obtained more realistic paths that passengers would take, namely they would go from station to station, rarely coming back to locations already visited. If the random walk found itself in an isolated path that only leads to a final destination of a tramline or back to where he was before, he would not get stuck on the way going forward and backwards repeatedly. From there if he still had time left, we allowed it to jump/teleport to a node he had not visited yet. We then iterated the random walks several times and took the sample average of the coverage for different finishing times. Moreover we also sampled which tram and bus stations were missed/visited the most. Indeed. the most visited nodes are those of the most known stations in Zurich, while the least visited are mostly final destinations at the corners of the city. Lastly we measured how the average amount of jumps/teleportations of the random walk would change for different finishing times.

Let us begin with explaining the mathematical ideas behind our work more in detail.

Theoretical framework

The concepts presented here are analgous to those defined in (3), with some minor changes. A **Multi-layered network** is a collection (W, C), where $W = (W^\alpha)_{\alpha=1}^L$ and for each $\alpha = 1, ..., L$, $W^\alpha = (W_{ij}^\alpha)_{i,j=1}^{N_\alpha}$ is a weighted adjacency matrix of a (single-layer) network with N_α nodes. $C = (C^{\alpha\beta})_{\alpha.\beta=1}^L$, with $C^{\alpha\beta} = (C_i^{\alpha\beta})_{i=1}^N \in \mathbb{R}_{\geq 0}^N$ is called the cost matrix ($N = \sum_{\alpha=1}^L N_\alpha$). It is such that $C_i^{\alpha\beta} = 0$ if i is not in one of the two layers α or β and > 0 else. The cost function of a multi layered network is related to how favourable it is to swap layers.

A (time dependent) random walk on a multi-layered random network is a discrete time stochastic process that can be identified by the transition kernel $T(t)$. The transition kernel is defined by the transition probabilities:

- $P_{ij}^{\alpha\alpha}(t)$, the probability to move to a node j in the layer α at time t
- $P_{ii}^{\alpha\alpha}(t)$, the probability to stay at a node i in the layer α at time t
- $P_{ii}^{\alpha\beta}(t)$, the probability at a node i to change from layer α to layer β at time t
- $P_{ij}^{\alpha\beta}(t)$, the probability at a node i in layer α to change layer and move to a node j layer β at time t.

In the case where $L = 2$ the transiton kernel $T(t)$ at time t is then given by the $N \times N$ matrix:

$$\begin{bmatrix} P_{11}^{11}(t) & \dots & P_{1N_\alpha}^{11}(t) & P_{11}^{12}(t) & \dots & P_{1N_\beta}^{12}(t) \\ P_{21}^{11}(t) & \dots & P_{2N_\alpha}^{11}(t) & P_{21}^{12}(t) & \dots & P_{2N_\beta}^{12}(t) \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ P_{N_\alpha 1}^{11}(t) & \dots & P_{N_\alpha N_\alpha}^{11}(t) & P_{N_\alpha 1}^{12}(t) & \dots & P_{N_\alpha N_\beta}^{12}(t) \\ P_{11}^{21}(t) & \dots & P_{1N_\alpha}^{21}(t) & P_{11}^{22}(t) & \dots & P_{1N_\beta}^{22}(t) \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ P_{N_\beta 1}^{21}(t) & \dots & P_{N_\beta N_\alpha}^{21}(t) & P_{N_\beta 1}^{22}(t) & \dots & P_{N_\beta N_\beta}^{22}(t) \end{bmatrix}$$

A necessary condition for matrix to be a transition kernel is that each row sums up to one. Each entry of a row i represents the probability of one of the options the random walk has at the node i.

Let $p_i(t)$ be the probability to find the random walk in the node $i = 1, ..., N$. We have that at time $t = 0$ the random walk can only possibly be at the starting location i_0, hence $p(0) = (p_i(0))_{i=1}^N = e_{i_0}$ where e_i is the i-th canonical vector in $\mathbb{R}^N$ in row form. Then if we consider that the random walk only depends on the transition kernel at time t we get that:

$$p(t+1) = p(t)T(t) = p(0)T(0)T(1)...T(t-1)T(t) \quad [1]$$

Hence we can get $p(t)$ recursively. Define now $\widetilde{\delta}_{ij}(t)$ to be the probability that we **never** find the random walk at node i after t time steps if it started at j. At time $t = 0$ we have that $\widetilde{\delta}_{ij}(0) = 0$ if $i = j$ and $\widetilde{\delta}_{ij}(0) = 1$ else, because at time 0 the random walk finds itself where it started. Using $p(t)$ we can write:

$$\widetilde{\delta}_{ij}(t) = (1 - p_j(0)) \prod_{s=1}^{t} (1 - p_i(t-s)) \quad [2]$$

The **coverage** of the network by the random walk is then given as the expected fraction of the nodes that have been visited after time t, which is equal to 1 minus the expected fraction of nodes not visited after time t. We can define it by using $\widetilde{\delta}_{ij}(t)$:

$$\rho(t) = 1 - \frac{1}{N^2} \sum_{i,j=1}^{N} \widetilde{\delta}_{ij}(t) \quad [3]$$

The recursive relation in 1 is very useful because as long we store the initial random walk distribution $p(0)$ and update p accordingly at each time step. Then we only need to compute the coverage at the final time and not repeatedly.

Construction of tram and bus network for Zurich

The first part was to construct to adjacency matrices W^1 and W^2 for the tram and and bus network separately. From now on layer 1 will always denote the tram network and layer 2 the bus network. Since the nodes represent the tram and bus stations of course there should be an edge between nodes if a bus or tram directly drives between those two stations. However we also wanted it to consider how many trams and busses go directly to which nodes and how many minutes does it take, considering that some stations are further apart or there might be more traffic in some locations of the city. For this we created excel files containing all the station paths for each tram line and bus line. This was accessible via the online tool of the VBZ (2). We took out the cases in which the paths included the final depot station after 23 p.m. and also excluded transportation lines that operate mostly outside of Zurich or are operated much less often than the rest (for example the tram line 5 only operates at weekends and was thus excluded). Additionally we used the expected time that each tram und bus line takes from each specific station to the next station, which can also be found on the online VBZ timetable. We then created two adjacency matrices for the tram and bus network each. The matrices A^1 and A^2 only encode how many tramlines and buslines resp. go from one station to another. For example if $A^1_{ij} = 2$ it means that there are two tramlines going from the tram station i to the tram station j. The matrices W^1 and W^2 additionally are weighted by how much time it takes to move between stations. So for example if $W^2_{ij} = 4$ its possible that there are two bus lines that need 2 minutes to go from bus station i to bus station j or only one bus line that needs 4 minutes. In figure 1 and 2 one can see the resulting network layers:

Here they were drawn as undirected unweighted graphs but

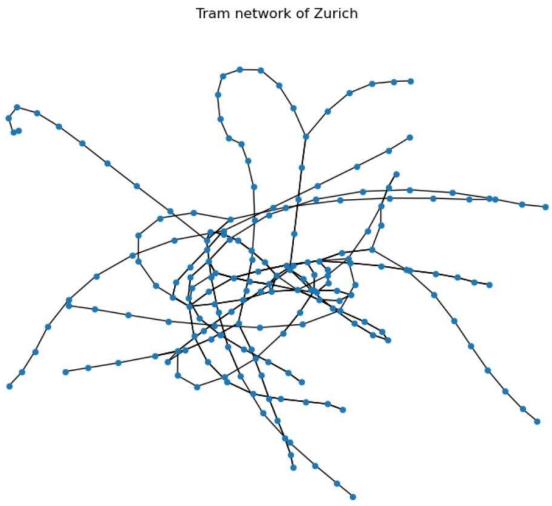

Fig. 1. Tram network of the city of Zurich

in the code attached one can find the multi graphs aswell (since for many stations there are more than one tram or bus line that connects them).

One notes the clear tree like shape that the public transport networks typically have.

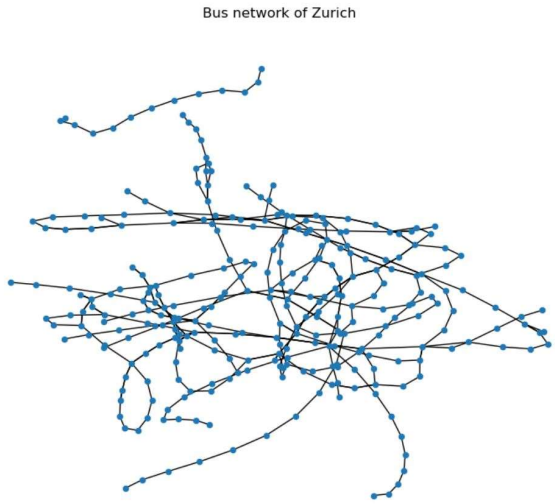

Fig. 2. Bus network of the city of Zurich

Transition kernel at time 0

Using these matrices we next constructed the transition kernel. We will consider the time step size to be always one minute. First we set the probabilities $P^{12}_{ij}(t) = P^{21}_{ij}(0) = 0, \forall t$ if $i \neq j$ because it is realistically not possible in only one minute to change from bus to tram or vice versa and also travel to a different station. Inside the layers we set $P^{\alpha\alpha}_{ij}(0) \propto A^{\alpha}_{ij}/W^{\alpha}_{ij}$ if $i \neq j$, as it is less likely to arrive at the next minute at a station that is further away but more likely if there are more tram or bus lines going there (here $\propto$ means equal up to a probability normalizing constant). On the other hand we set

$$P^{\alpha\alpha}_{ii}(0) \propto c \cdot (k^{\alpha}_i)^2$$

where C is a parameter and $k^{\alpha}_i = \sum_{j=1}^{N_\alpha} A^{\alpha}_{ij}$ is how many tram lines or bus lines respectively leave the station i. This is supposed to encode a much higher likeliness to still be at the same station the next minute, if at this station there many different tram or bus lines stopping, as one might want to change from line inside the layer aswell. We then adapted the parameterr c after running a few random walk simulations, to fit a reasonable amount of changes inside the different layers. For the probability of changing layer at a station $P^{\alpha\beta}_{ij}$ we set it to 0 if at that station either there are no tram lines or there are no bus lines. If a station indeed has both, consider i_α to be the index in the node list of layer α and i_β to be index in layer β (they are not going to be the same due to the two networks having nodes that do not exist in the other network and also not sharing the same amount). We then take $P^{\alpha\beta}_{i_\alpha i_\beta}(0) \propto k^{\beta}_i/k^{\alpha}_i$, which is supposed to model the behaviour already explained in the introduction.

In figure 3 we printed an example of how the distribution looks like for the station Milchbuck, if the random walk would find itself currently inside the tram layer. The low probability of $\approx 7.6\%$ to switch to the bus layer is due to the fact that there are 4 tram lines stopping at Milchbuck vs only 3 bus lines, but also due to Milchbuck having four neighbors inside the tram layer.

```
prob of staying here is: 0.0759602934829521
prob of switching to a bus is: 0.08333333333333333
prob of going to Hirschwiesenstrasse is: 0.19781326427852108
prob of going to Guggachstrasse is: 0.2967198964177817
prob of going to Universität Irchel is: 0.19781326427852108
prob of going to Tierspital is: 0.14835994820889084
```

Fig. 3. Probabilities for next steps from Milchbuck (tram layer)

Random walk simulation

With the transition kernel at the start time defined we can code the random walk simulation. Firstly we fix a finishing time t_{end}, at which we stop the random walk and we choose the starting node. There are 197 tram stops and 293 bus stops resulting in a total of 490 possible starting points. If we would not distinguish between tram and bus stop, then there are only 410, as 80 stations have both bus and trams passing through. We pick an arbitrary index i between 1 and 490 (in python between 0 and 489) to choose the starting station. If i is less than 198 (or 197 in python) the random walk starts inside the tram network and if it is larger or equal to 198 it starts inside the bus network, no matter if the node i corresponds to a station that has both bus and tram stops. Now at the station i the random walk can move to any neighbor inside the layer or switch layer if the station the node represents is both in the tram and the bus network. The probabilities for these events is given by the i-th row of the transition kernel at time 0 defined before. If the random walk mmoves to a neighbor, the transition kernel stays the same as before and we repeat the step at the new node. However lets suppose the random walk started at layer 1 (tram) and now switched to layer 2 (bus) and let i_1 be the node index in the tram network and i_2 be the node index in the bus network. Then the transition kernel $T(1)$ is changed compared to $T(0)$ by setting $P^{21}_{i_2 i_1}(1) = 0$ to prevent it from switching back immediately or at any later time. The remaining entries of the row are then renormalized, so that they sum up to 1 again. All the other entries of the kernel do not change. Before updating the kernel we compute and store $p(1) = p(0)T(0)$ to later compute the coverage by using 1, 2 and 3. This is repeated at each time step until time finishing time t_{end}.

After the first few test runs we realized the problem with the tree like shape of the networks, thus at each time t we checked if the new node in the random walk is the same as $t-2$ or $t-3$ (remember he is also allowed to stay at a node without switching layer). In that case we repeated the step without updating the time to $t+1$. This became very computationally expensive as we used a self calling function to do so. Especially problematic were the final destinations of tram and bus lines that are not connected to any other stations, as then the function calls itself infinitely often. To prevent this we just checked if the next node is a final destionation and if it has less then three neighbors. If this was true, we first decided to just stop the random walk and print a message that the random walk had stopped at his location. However the problem became that when increasing the finishing time t_{end} to higher values (≥ 100) the random walk would almost always stop far earlier than t_{end} as soon as it entered an isolated path as one can see in figures 1 and 2. This was not very practical to study the coverage of the network, this we decided to allow the random walk to jump

```
Starting point: Technopark (tram)
Switched from tram to bus at: Schiffbau
Switched from bus to tram at: Schaufelbergerstrasse
Switched from tram to bus at: Triemli
Arrived at end stop: Triemli
jumped to: Schaffhauserplatz
Switched from tram to bus at: Schaffhauserplatz
Switched from bus to tram at: Seilbahn Rigiblick
amount of jumps: 1
['Technopark', 'Technopark', 'Technopark', 'Schiffbau', 'Schiffbau', 'Bahnhof Hardbrücke', 'Hardplatz', 'Hardplatz', 'Herdernst
rasse', 'Herdernstrasse', 'SBB-Werkstätte', 'SBB-Werkstätte', 'Letzipark', 'Letzipark', 'Letzibach', 'Letzibach', 'Letzipark We
st', 'Baslerstrasse', 'Baslerstrasse', 'Bristenstrasse', 'Bahnhof Altstetten', 'Bahnhof Altstetten', 'Seidelhof', 'Seidelhof',
'Bahnhof Altstetten', 'Solidapark', 'Solidapark', 'Bahnhof Altstetten', 'Letzibach', 'Letzipark West', 'Kappeli', 'Kappeli', 'F
lurstrasse', 'Flurstrasse', 'Albisrank', 'Albisrank', 'Hubertus', 'Im Gut', 'Im Gut', 'Schaufelbergerstrasse', 'Schaufelbergers
trasse', 'Triemli', 'Triemli', 'Schaffhauserplatz', 'Schaffhauserplatz', 'Scheuchzerstrasse', 'Scheuchzerstrasse', 'Seilbahn Ri
giblick', 'Seilbahn Rigiblick', 'Winkelriedstrasse', 'Haldenbach', 'Haldenbach', 'ETH/Universitätsspital', 'Haldenegg']
```

Fig. 4. Example of random walk path

to a node the random walk has not been visited yet, with a degree higher than 3 so that it does not find itself quickly stuck again. In figure 4 one can see an example of a path the random walk takes for $t_{end} = 50$. The simulation code prints out additionally at which station the random walk switched between tram or bus, if he arrived at a final destination and at which one, and to which node it jumped next. Finally it also prints how many jumps it took during the run.

Results

First we will show what the result for the coverage was, depending on how large the finishing time was. We note that because the transition kernels $T(0), ..., T(t_{end})$ for different iterations of the random walk are not the same, becuase the rows that are changed depend on where the random walked switched layers, and the paths are completely randomized for each iteration. This results in a different coverage for different iterations, and is the reason why we took a sample mean to approximate the coverage. We sampled 50 iterations of a random walk for each of the finishing times $t_{end} = 10, 50, 100, 500, 1000, 1500$. For higher values of t_{end} it would too often occur that in case it found itself at a final destination it would run out of remaining nodes to jump to that have a high enough degree, hence higher values were not estimated. We additionally sampled the amount of time the random walk would do such jumps after arriving at final destinations of the network and took the average, which we will denote by $\bar{J}(t_{end})$. In table 1 one can see the results containing also the variance of the samples σ_ρ and σ_J:

Table 1. Average of sample for coverage and amount of jumps for different finishing times

t_{end}	$\bar{\rho}(t_{end})$	σ_ρ	$\bar{J}(t_{end})$	σ_J
10	2.05%	0.0001	0.72	0.68
50	6.28%	0.0005	2.54	2.33
100	11.27%	0.0011	5.46	6.49
500	41.96 %	0.0034	25.78	33.77
1000	66.10%	0.0022	40.67	36.58
1500	78.61%	0.0001	78.6	64.12

First column is the time t_{end} at which the random walk ends. Second column shows on average how much percent of the the network is being covered by a random walk with finishing time t_{end}. To the right of it the variance of the 50 value long sample of coverages. In the fourth column is the average amount of times the random walk was forced to jump to a new location and in the column to the right of it the variance of the amount of jumps of the sample.

Unsurprisingly we can see that with more time the random walk can cover more and more of the network, which means that our model is indeed working as we hoped. The jumps at final destinations of the transport lines also do become more frequent, however due to the high variance compared to the mean, it seems that it is very different for each iteration of the random walk. Lastly we wanted to verify if the most visited tram and bus stations of our model are indeed those we would expect. Some of the main public transport hubs of Zurich include Milchbuck, Schaffhauserplatz, Bucheggplatz and stops that are next to Zürich HB, the train main station of Zurich. So if our random walk should imitate the public transport flow of Zurich, those above named station should be missed rather rarely. We iterated 50 times again the random walk with finishing time $t_{end} = 500$ and always stored which stations were not visited from the total list of tram and bus stations in Zurich. We then measured the frequency of how often each station was missed during an iteration. Considering that for $t_{end} = 500$ on average 41.95% of the entire network is covered, the chance of one node to never be missed during those 50 iterations is $\approx (0.4)^{50}$, so rather unlikely. Below one can see part of the rankings; the smaller the frequency with which the station was missed the higher it is in the ranking:

	Station	Percentage of how often missed
0	Schaffhauserplatz	0.010
1	Central	0.010
2	Kunsthaus	0.010
3	Schwert	0.010
4	Bahnhofplatz/HB	0.010
5	Bucheggplatz	0.010
6	Rosengartenstrasse	0.010
7	Stampfenbachplatz	0.010
8	Escher-Wyss-Platz	0.020
9	Bahnhofquai/HB	0.020
10	Limmatplatz	0.031

Fig. 5. Top 10 least missed/most visited stations by the random walk

400	Geeringstrasse	0.451
401	Holzerhurd	0.461
402	Zürich Flughafen Fracht	0.461
403	Werdhölzli	0.461
404	Burgwies	0.461
405	Wetlistrasse	0.471
406	Hermetschloo	0.492
407	Triemlispital	0.492
408	Hedwigsteig	0.492
409	Rütihof	0.512

Fig. 6. Bottom 10 least missed/most visited stations by the random walk

The full rankings can be found in the jupyter notebook code. We did the same for finishing times $t_{end} = 1000$ and one $t_{end} = 100$. The lower and upper end of the rankings are very similiar, what differs is the percentage of times the stations are missed. But this is due to the fact that for those finishing times much more / much less of the network is covered on average. In figure 5 one can see that many of the stations we mentioned are in the top 10 (Milchbuck is at position 12). Central for example is a station close to the Zürich HB at which 7 tram lines and 2 bus line stop. The only surprising appearance is Schwert (2 bus lines, 1 tram line), but it could be explained if Schwert has a high betweenness centrality for example.

Which stations do we expect to be the least frequently visited nodes? They should be stations that are very isolated from other stations, so final destinations of bus and tram lines that lie towards the city outskirts for example or stations where only one bus or tram line passes by. And indeed this is the case: Rütihof is the final destination of bus line 46 and cannot be reached with another transportation line. Triemlispital is the same but for bus line 80. Hedwigsteig is not a final destination but can only be reached by the tram line 11.

Conclusion

We have extended the concept of random walks on two layered networks for transportation networks, with a more sophisticated cost for the switching of layers. The random walk implemented here that is partially time dependent successfully covers a decent percentage of the network in considerably small time scales, and also moves in a much more realistic pattern than it would for the usual random walk that is a Markov process. The code was very much adapted to the Zurich bus and tram transportation network, however it would not be too much work to adapt this to a two-layer network of a different city. All that would be necessary are datasets with the paths of the transportation lines. The transition kernel could then be constructed analogously, and the coded class "Simulation" could be reused for this.

The most logical extension to this work, would be to add a third layer to this network, for example Zurich has many train stations with train running at a half an hour frequency. It should be remarked that many larger cities also have three layer transportation networks. London for example has an underground, overground and bus network. Some critique to our model could be that it makes covering the network for the random walk too fast by allowing it to jump to unvisited locations after arriving at isolated final destinations. One way to counteract this could be instead of "teleporting" the randomm walk allow it to walk back to the node two time steps ago if in between he reached a final destination.

ACKNOWLEDGEMENTS: I thank Samuel Koovely, Rathes Sriram and Otto Buck for taking their time and helping me implement my ideas for the random walk model and overcome frustrating coding errors.

1. M Newman, *Networks, 2nd Edition.* (Oxford University Press), (2018).
2. VO stop time tables. (Stadt Zürich), (2024).
3. SG Manlio De Domenico, Albert Solé-Ribalta, A Arenas, Navigability of interconnected networks under random failures. *Proc. Natl. Acad. Sci.* **111**, 8351–8356 (2014).
4. M chain. (Wikipedia), (https://en.wikipedia.org/wiki/Markovchain).